

My Project

Generated by Doxygen 1.15.0

1 File Index	1
1.1 File List	1
2 File Documentation	3
2.1 ConditionalRandomField.py File Reference	3
2.1.1 Detailed Description	4
2.1.2 Function Documentation	4
2.1.2.1 apply_dense_crf()	4
2.1.2.2 calculate_dice_coefficient()	5
2.1.2.3 evaluate_segmentation()	5
2.1.2.4 extract_features_with_coords()	6
2.1.2.5 get_probability_map()	6
2.1.2.6 main()	7
2.1.2.7 map_labels_back()	7
2.1.2.8 predict_with_crf()	8
2.1.2.9 remap_mask_labels()	8
2.1.2.10 resize_image()	9
2.1.2.11 resize_mask()	9
2.1.2.12 save_comparison_metrics_as_image()	10
2.1.2.13 save_summary_metrics_as_image()	10
2.1.2.14 train_naive_bayes_memory_efficient()	11
2.1.2.15 visualize_crf_comparison()	11
Index	13

Chapter 1

File Index

1.1 File List

Here is a list of all documented files with brief descriptions:

[ConditionalRandomField.py](#)

Optic disc and cup segmentation using Naive Bayes classifier with Dense CRF refinement . . . 3

Chapter 2

File Documentation

2.1 ConditionalRandomField.py File Reference

Optic disc and cup segmentation using Naive Bayes classifier with Dense CRF refinement.

Functions

- [ConditionalRandomField.resize_image](#) (image, max_size=512)
Resize regular fundus image to reduce memory usage.
- [ConditionalRandomField.resize_mask](#) (mask, max_size=512)
Resize segmentation mask using nearest neighbor interpolation.
- [ConditionalRandomField.remap_mask_labels](#) (mask)
Remap mask pixel values to standardized label values.
- [ConditionalRandomField.extract_features_with_coords](#) (image, normalize_coords=True, use_both_colorspaces=True)
Extract color and spatial features from fundus image.
- [ConditionalRandomField.train_naive_bayes_memory_efficient](#) (fundus_files, mask_files, use_both_colorspaces=True, batch_size=5, max_size=512)
Train Naive Bayes classifier using batch processing for memory efficiency.
- [ConditionalRandomField.get_probability_map](#) (clf, scaler, image, use_both_colorspaces=True)
Generate probability map from Naive Bayes classifier predictions.
- [ConditionalRandomField.apply_dense_crf](#) (image, probabilities, n_iters=5, sxy_gaussian=3, compat_gaussian=3, sxy_bilateral=80, srgb_bilateral=13, compat_bilateral=10)
Apply Dense Conditional Random Field refinement to probability map.
- [ConditionalRandomField.map_labels_back](#) (crf_output, label_mapping)
Map CRF integer labels back to original mask values.
- [ConditionalRandomField.predict_with_crf](#) (clf, scaler, test_image, use_both_colorspaces=True, apply_crf=True, crf_params=None)
Predict segmentation with optional CRF refinement.
- [ConditionalRandomField.calculate_dice_coefficient](#) (pred, gt, label)
Calculate Dice coefficient for a specific segmentation class.
- [ConditionalRandomField.evaluate_segmentation](#) (pred_mask, gt_mask)
Evaluate segmentation performance using multiple metrics.
- [ConditionalRandomField.save_comparison_metrics_as_image](#) (metrics_nb, metrics_crf, image_name, save_path)
Save comparison table of Naive Bayes and CRF metrics as image.

- [ConditionalRandomField.save_summary_metrics_as_image](#) (metrics_nb_list, metrics_crf_list, save_path)
Save overall summary metrics across all test images as table image.
- [ConditionalRandomField.visualize_crf_comparison](#) (original, naive_mask, crf_mask, gt_mask=None, save_path=None)
Visualize comparison between Naive Bayes and CRF segmentation results.
- [ConditionalRandomField.main](#) ()
Memory-efficient main execution pipeline with CRF refinement.

2.1.1 Detailed Description

Optic disc and cup segmentation using Naive Bayes classifier with Dense CRF refinement.

This module implements a memory-efficient pipeline for retinal image segmentation that combines Naive Bayes classification with Dense Conditional Random Field (CRF) post-processing to segment optic disc and cup regions from fundus images.

Author

Rishab Ramesh Nair

Date

November 2, 2025

2.1.2 Function Documentation

2.1.2.1 `apply_dense_crf()`

```
ConditionalRandomField.apply_dense_crf (
    image,
    probabilities,
    n_iters = 5,
    sxy_gaussian = 3,
    compat_gaussian = 3,
    sxy_bilateral = 80,
    srgb_bilateral = 13,
    compat_bilateral = 10)
```

Apply Dense Conditional Random Field refinement to probability map.

Refines segmentation predictions using Dense CRF with Gaussian and bilateral pairwise potentials to enforce spatial smoothness and edge-aware consistency.

Parameters

<i>image</i>	Input RGB image for bilateral filtering.
<i>probabilities</i>	Class probability array of shape (n_classes, height, width).
<i>n_iters</i>	Number of CRF inference iterations. Default is 5.

<i>sxy_gaussian</i>	Gaussian kernel spatial standard deviation. Default is 3.
<i>compat_gaussian</i>	Gaussian kernel compatibility weight. Default is 3.
<i>sxy_bilateral</i>	Bilateral kernel spatial standard deviation. Default is 80.
<i>srgb_bilateral</i>	Bilateral kernel color standard deviation. Default is 13.
<i>compat_bilateral</i>	Bilateral kernel compatibility weight. Default is 10.

Returns

Refined segmentation map of shape (height, width) with integer class labels.

Apply Dense CRF refinement

2.1.2.2 calculate_dice_coefficient()

```
ConditionalRandomField.calculate_dice_coefficient (
    pred,
    gt,
    label)
```

Calculate Dice coefficient for a specific segmentation class.

Computes the Dice similarity coefficient (F1 score) between predicted and ground truth segmentation for a single class label.

Parameters

<i>pred</i>	Predicted segmentation mask.
<i>gt</i>	Ground truth segmentation mask.
<i>label</i>	Class label value to evaluate.

Returns

Dice coefficient value in range [0, 1] where 1 is perfect overlap.

Calculate Dice coefficient for a specific class

2.1.2.3 evaluate_segmentation()

```
ConditionalRandomField.evaluate_segmentation (
    pred_mask,
    gt_mask)
```

Evaluate segmentation performance using multiple metrics.

Computes accuracy and Dice coefficients for all segmentation classes (disc, cup, background) and returns comprehensive evaluation metrics.

Parameters

<i>pred_mask</i>	Predicted segmentation mask.
<i>gt_mask</i>	Ground truth segmentation mask.

Returns

Dictionary containing 'accuracy', 'dice_disc', 'dice_cup', 'dice_background', and 'dice_mean' metrics.

Evaluate segmentation performance

2.1.2.4 extract_features_with_coords()

```
ConditionalRandomField.extract_features_with_coords (
    image,
    normalize_coords = True,
    use_both_colorspaces = True)
```

Extract color and spatial features from fundus image.

Extracts pixel-wise features including RGB/HSV color information and normalized spatial coordinates for training or prediction.

Parameters

<i>image</i>	Input fundus image (BGR format).
<i>normalize_coords</i>	If True, normalize coordinates to [0,1] range. Default is True.
<i>use_both_colorspaces</i>	If True, extract both RGB and HSV features. Default is True.

Returns

Feature array of shape (height*width, n_features) where n_features is 8 (RGB + HSV + x + y) or 5 (RGB + x + y) depending on use_both_colorspaces.

Extract color and spatial features from fundus image

2.1.2.5 get_probability_map()

```
ConditionalRandomField.get_probability_map (
    clf,
    scaler,
    image,
    use_both_colorspaces = True)
```

Generate probability map from Naive Bayes classifier predictions.

Computes per-pixel class probabilities for all segmentation classes using the trained Naive Bayes classifier.

Parameters

<i>clf</i>	Trained GaussianNB classifier.
<i>scaler</i>	Fitted StandardScaler for feature normalization.
<i>image</i>	Input fundus image (BGR format).
<i>use_both_colorspaces</i>	If True, use both RGB and HSV features. Default is True.

Returns

Probability array of shape (n_classes, height, width) with values in [0,1].

Get probability map from Naive Bayes classifier

2.1.2.6 main()

ConditionalRandomField.main ()

Memory-efficient main execution pipeline with CRF refinement.

Executes the complete segmentation pipeline including:

- Loading and preprocessing training data
- Training Naive Bayes classifier with batch processing
- Testing on validation images
- Applying CRF refinement
- Computing evaluation metrics
- Saving visualizations and metric comparisons

Returns

None. Saves all outputs to the configured OUTPUT_DIR.

Memory-efficient main execution pipeline with CRF

2.1.2.7 map_labels_back()

```
ConditionalRandomField.map_labels_back (
    crf_output,
    label_mapping)
```

Map CRF integer labels back to original mask values.

Converts class indices from CRF output back to the original grayscale mask values (0, 128, 255).

Parameters

<i>crf_output</i>	CRF output with integer class labels.
<i>label_mapping</i>	Dictionary mapping integer labels to original mask values.

Returns

Final segmentation mask with original label values.

Map CRF integer labels back to original mask values

2.1.2.8 predict_with_crf()

```
ConditionalRandomField.predict_with_crf (
    clf,
    scaler,
    test_image,
    use_both_colorspaces = True,
    apply_crf = True,
    crf_params = None)
```

Predict segmentation with optional CRF refinement.

Generates segmentation predictions using Naive Bayes classifier and optionally applies Dense CRF post-processing for refinement.

Parameters

<i>clf</i>	Trained GaussianNB classifier.
<i>scaler</i>	Fitted StandardScaler for feature normalization.
<i>test_image</i>	Input test fundus image (BGR format).
<i>use_both_colorspaces</i>	If True, use both RGB and HSV features. Default is True.
<i>apply_crf</i>	If True, apply CRF refinement. Default is True.
<i>crf_params</i>	Dictionary of CRF parameters. If None, uses default values.

Returns

Tuple of (final_mask, naive_bayes_mask) where final_mask is the CRF-refined result (or Naive Bayes result if apply_crf=False) and naive_bayes_mask is the unrefined classifier output.

Predict segmentation with optional CRF refinement

2.1.2.9 remap_mask_labels()

```
ConditionalRandomField.remap_mask_labels (
    mask)
```

Remap mask pixel values to standardized label values.

Converts grayscale mask values to exactly three discrete labels: 0 (disc), 128 (cup), and 255 (background) based on intensity thresholds.

Parameters

<i>mask</i>	Input grayscale mask with arbitrary intensity values.
-------------	---

Returns

Remapped mask with values in {0, 128, 255}.

Remap mask values to exactly 0, 128, or 255

2.1.2.10 resize_image()

```
ConditionalRandomField.resize_image (
    image,
    max_size = 512)
```

Resize regular fundus image to reduce memory usage.

Proportionally resizes an image so that its largest dimension does not exceed the specified maximum size while maintaining aspect ratio.

Parameters

<i>image</i>	Input image array (BGR format).
<i>max_size</i>	Maximum dimension size in pixels. Default is 512.

Returns

Resized image or original image if already within size limit.

Resize regular image (fundus) to reduce memory usage

2.1.2.11 resize_mask()

```
ConditionalRandomField.resize_mask (
    mask,
    max_size = 512)
```

Resize segmentation mask using nearest neighbor interpolation.

Resizes a mask image while preserving discrete label values by using nearest neighbor interpolation instead of bilinear or bicubic methods.

Parameters

<i>mask</i>	Input mask array with discrete labels.
<i>max_size</i>	Maximum dimension size in pixels. Default is 512.

Returns

Resized mask or original mask if already within size limit.

Resize mask using nearest neighbor to preserve discrete labels

2.1.2.12 `save_comparison_metrics_as_image()`

```
ConditionalRandomField.save_comparison_metrics_as_image (
    metrics_nb,
    metrics_crf,
    image_name,
    save_path)
```

Save comparison table of Naive Bayes and CRF metrics as image.

Creates and saves a formatted table comparing segmentation metrics between Naive Bayes and CRF-refined predictions for a single test image.

Parameters

<i>metrics_nb</i>	Dictionary of Naive Bayes metrics.
<i>metrics_crf</i>	Dictionary of CRF-refined metrics.
<i>image_name</i>	Name of the test image.
<i>save_path</i>	File path where the metrics comparison image will be saved.

Returns

None. Saves image to disk.

Save comparison of Naive Bayes and CRF metrics as an image

2.1.2.13 `save_summary_metrics_as_image()`

```
ConditionalRandomField.save_summary_metrics_as_image (
    metrics_nb_list,
    metrics_crf_list,
    save_path)
```

Save overall summary metrics across all test images as table image.

Computes average metrics across multiple test images and saves a formatted comparison table showing overall performance improvements with CRF.

Parameters

<i>metrics_nb_list</i>	List of Naive Bayes metrics dictionaries for all test images.
<i>metrics_crf_list</i>	List of CRF metrics dictionaries for all test images.
<i>save_path</i>	File path where the summary metrics image will be saved.

Returns

None. Saves image to disk.

Save overall summary metrics as a table image

2.1.2.14 train_naive_bayes_memory_efficient()

```
ConditionalRandomField.train_naive_bayes_memory_efficient (
    fundus_files,
    mask_files,
    use_both_colorspaces = True,
    batch_size = 5,
    max_size = 512)
```

Train Naive Bayes classifier using batch processing for memory efficiency.

Trains a Gaussian Naive Bayes classifier on fundus images and their segmentation masks using incremental learning with batch processing to reduce memory usage.

Parameters

<i>fundus_files</i>	List of file paths to training fundus images.
<i>mask_files</i>	List of file paths to corresponding segmentation masks.
<i>use_both_colorspaces</i>	If True, use both RGB and HSV color spaces. Default is True.
<i>batch_size</i>	Number of images to process per batch. Default is 5.
<i>max_size</i>	Maximum image dimension for resizing. Default is 512.

Returns

Tuple of (trained_classifier, fitted_scaler) where classifier is GaussianNB and scaler is StandardScaler for feature normalization.

Memory-efficient training with batching

2.1.2.15 visualize_crf_comparison()

```
ConditionalRandomField.visualize_crf_comparison (
    original,
    naive_mask,
    crf_mask,
    gt_mask = None,
    save_path = None)
```

Visualize comparison between Naive Bayes and CRF segmentation results.

Creates a side-by-side visualization comparing original image, ground truth, Naive Bayes prediction, and CRF-refined prediction.

Parameters

<i>original</i>	Original fundus image (BGR format).
<i>naive_mask</i>	Naive Bayes segmentation mask.
<i>crf_mask</i>	CRF-refined segmentation mask.

<i>gt_mask</i>	Ground truth mask. If None, only shows 3 panels. Default is None.
<i>save_path</i>	File path to save visualization. If None, displays instead. Default is None.

Returns

None. Either saves or displays the visualization.

Visualize comparison between Naive Bayes and CRF refinement

Index

`apply_dense_crf`
 ConditionalRandomField.py, [4](#)

`calculate_dice_coefficient`
 ConditionalRandomField.py, [5](#)

`ConditionalRandomField.py`, [3](#)
 [apply_dense_crf](#), [4](#)
 [calculate_dice_coefficient](#), [5](#)
 [evaluate_segmentation](#), [5](#)
 [extract_features_with_coords](#), [6](#)
 [get_probability_map](#), [6](#)
 [main](#), [7](#)
 [map_labels_back](#), [7](#)
 [predict_with_crf](#), [8](#)
 [remap_mask_labels](#), [8](#)
 [resize_image](#), [9](#)
 [resize_mask](#), [9](#)
 [save_comparison_metrics_as_image](#), [9](#)
 [save_summary_metrics_as_image](#), [10](#)
 [train_naive_bayes_memory_efficient](#), [10](#)
 [visualize_crf_comparison](#), [11](#)

`evaluate_segmentation`
 ConditionalRandomField.py, [5](#)

`extract_features_with_coords`
 ConditionalRandomField.py, [6](#)

`get_probability_map`
 ConditionalRandomField.py, [6](#)

`main`
 ConditionalRandomField.py, [7](#)

`map_labels_back`
 ConditionalRandomField.py, [7](#)

`predict_with_crf`
 ConditionalRandomField.py, [8](#)

`remap_mask_labels`
 ConditionalRandomField.py, [8](#)

`resize_image`
 ConditionalRandomField.py, [9](#)

`resize_mask`
 ConditionalRandomField.py, [9](#)

`save_comparison_metrics_as_image`
 ConditionalRandomField.py, [9](#)

`save_summary_metrics_as_image`
 ConditionalRandomField.py, [10](#)

`train_naive_bayes_memory_efficient`